Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding for Augmented Reality (AR) Applications

TGS Ref No: TGS-2022017519

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 9.1

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
8.0	4 July 2026	Legacy trainer-slide-aligned course material.	Tertiary Infotech Academy
9.0	11 September 2026	Rebuilt as highly technical AI vibe-coding AR/VR courseware with localhost WebXR studio, eight labs and aligned assessments.	Dr Alfred Ang
9.1	12 September 2026	Added the live learner-facing AR/VR Playground across the deck, guide and labs.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Course Information **6**
Learning Outcomes and TSC Mapping **6**
Knowledge and Abilities 6
AR/VR Playground **6**
Install and run 7
Technical Reference Chapters **8**
Topic 1: AR User Interface Design and Storyboarding with AI Vibe Coding **8**
Requirement to scene contract 8
Data boundary for coding agents 9
Prompt contract anatomy 9
Supervised vibe-coding loop 9
Storyboard as a state machine 10
World-space versus screen-space UI 10
Angular size and viewing distance 10
Contrast over a live camera feed 10
Depth ordering and occlusion 11
Reachability and target bounds 11
Equipment specifications into AR content 11
Coordinate systems and metric units 12
Device capability decision matrix 12
Secure context and permission intent 12
XR session lifecycle 13
Versioned scene schema 13
Topic 2: AR Application Development with Localhost WebXR **13**
Three.js renderer ownership 13
Scene graph transform chain 14
Perspective camera projection 14
Lighting model and evidence 14
Material contract 15
Geometry and draw-call cost 15
Metric scale and asset normalisation 15
Delta-time animation 15
Desktop orbit fallback 16
AR session feature request 16
Viewer-space hit-test source 16
Reticle as placement evidence 17
Anchor and persistence boundary 17

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Raycast selection pipeline....17
Pointer coordinate normalisation....17
XR controller input sources....18
Scene regeneration and disposal....18
Scene export and import boundary....18
Topic 3: Testing, Reviewing and Optimising AR/VR Applications....19
AR and VR runtime boundary....19
Reference-space choice....19
Stereo frame budget....19
Controller rays and affordance....20
Teleport locomotion....20
Guardian and physical safety....20
Motion comfort controls....21
Spatial accessibility....21
FPS measurement....21
Draw-call and triangle budgets....21
Memory-leak diagnosis....22
Console and network evidence....22
Responsive control surface....22
Runtime defect taxonomy....23
Root-cause review loop....23
Regression-test pyramid....23
Developer handover contract....24
Detailed Lab Guides....24
Lab 01 — Scene Brief and Prompt Contract....24
Before you start....25
Detailed procedure....25
Test it....26
Troubleshooting....26
Cleanup....26
Lab 02 — Storyboard and Spatial UI....26
Before you start....27
Detailed procedure....27
Test it....28
Troubleshooting....28
Cleanup....28
Lab 03 — WebXR Capability and Equipment Gate....28
Before you start....29

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Detailed procedure....29
Test it....30
Troubleshooting....30
Cleanup....30
Lab 04 — Scene Graph, Lighting and Animation....30
Before you start....31
Detailed procedure....31
Test it....32
Troubleshooting....32
Cleanup....32
Lab 05 — Plane Detection, Hit Test and Anchors....32
Before you start....33
Detailed procedure....33
Test it....34
Troubleshooting....34
Cleanup....34
Lab 06 — Raycast, Touch and Spatial Feedback....34
Before you start....35
Detailed procedure....35
Test it....36
Troubleshooting....36
Cleanup....36
Lab 07 — Immersive VR, Controllers and Comfort....36
Before you start....37
Detailed procedure....37
Test it....38
Troubleshooting....38
Cleanup....38
Lab 08 — Runtime Review, Optimisation and Handover....38
Before you start....39
Detailed procedure....39
Test it....40
Troubleshooting....40
Cleanup....40
Assessment Preparation....40
Source Register....40

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Course Information

Course title	AI Vibe Coding for Augmented Reality (AR) Applications
WSQ course code	TGS-2022017519
TSC	Augmented Reality Application · PRE-CTS-3001-1.1
Duration	2 days · 16 hours
Provider	Tertiary Infotech Academy Pte Ltd · UEN 201200696W
Live Playground	https://alfredang.github.io/arvr-playground/
Local application	http://127.0.0.1:4173/

Official course page: https://www.tertiarycourses.com.sg/wsq-ai-vibe-coding-for-augmented-reality-ar-applications.html

Learning Outcomes and TSC Mapping

- LO1: Translate specifications and write storyboards for AR applications using spatial user-interface principles.
- LO2: Apply AR working principles to create procedures and review AR runtimes for usability and clarity.
- LO3: Overlay virtual objects on the real world, distinguish AR from VR, and advise developers on effective adjustments.

Knowledge and Abilities

Code	Requirement	Where practised
K1	Storyboarding for AR applications	Lab 2
K2	Principles of user interface design in AR	Lab 2, Lab 6
K3	Specifications and information on equipment, tools and/or machinery	Lab 1, Lab 3
K4	Instructions for work procedures	Lab 4
K5	Working principles of AR	Lab 3, Lab 4, Lab 5
K6	Concept of overlaying virtual objects onto the real world	Lab 5
K7	Distinct differences between virtual reality and augmented reality	Lab 7
A1	Translate work requirements, instructions and equipment specifications into AR content	Lab 1

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A2	Write storyboards that transfer work information into AR content	Lab 2
A3	Review AR runtimes to identify content errors and opportunities to improve usability and clarity	Lab 6, Lab 8
A4	Advise application developers on adjustments that improve AR effectiveness	Lab 8

AR/VR Playground

The shared browser application converts a constrained natural-language request into a reviewable scene contract, renders a Three.js desktop preview, exposes AR/VR launch capability, and reports validation evidence. Prompts are parsed locally in the browser.

Try the live Playground — no installation required: https://alfredang.github.io/arvr-playground/

Install and run

```
cd webapp

npm install --legacy-peer-deps

npm audit --audit-level=moderate

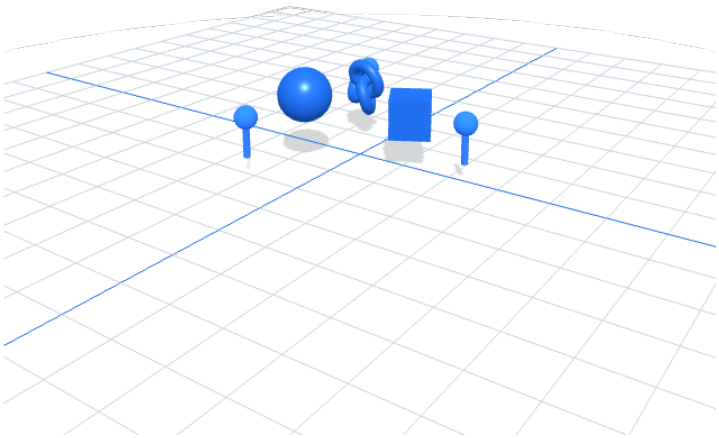
npm test

npm run dev
```

Open the local studio: http://127.0.0.1:4173/

> **Capability boundary:** Desktop preview works in a modern browser. Immersive AR/VR requires a supported WebXR device/browser and a secure context. Do not claim physical XR testing without actual device evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Technical Reference Chapters

Topic 1: AR User Interface Design and Storyboarding with AI Vibe Coding

Requirements · prompt contracts · storyboard states · spatial UI · capability gates

Alignment: K1 · K2 · K3 · A1 · A2

Requirement to scene contract

Every request becomes explicit targetMode, tracking, metric scale, content, interaction, performance and accessibility fields.

01	02	03	04
Work task	Equipment facts	Spatial content	Acceptance evidence

```
const scene = parsePrompt(requirement, modeOverride);
```

Failure mode: A vague prompt produces an attractive demo that cannot be assessed or safely handed over.
Control and recovery: Require a versioned schema and reject any generated change that cannot be traced to one field.
Verification evidence: A reviewer can point from each requirement to one JSON field

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

and one visible runtime state.

Data boundary for coding agents

Prompts may include public specifications and local source excerpts; credentials, learner records and proprietary data remain outside the agent context.

01	02	03	04
Classify input	Remove secrets	Minimise context	Inspect output

```
git diff -- webapp/src && npm test
```

Failure mode: A prompt includes an API key or personal record and the resulting transcript becomes a new disclosure surface.
Control and recovery: Use `.env.example`, synthetic data and named-file diffs; scan before commit.
Verification evidence: The prompt and diff contain no secret pattern, personal data or unrelated file change.

Prompt contract anatomy

A bounded coding prompt states the repository, files to inspect, behaviour to preserve, smallest intended change and exact tests.

01	02	03	04
Role and scope	Invariants	Requested delta	Verification

```
Read first. Do not edit yet. Return plan, risks, files and tests.
```

Failure mode: The agent rewrites working code, changes dependencies and claims success from syntax alone.
Control and recovery: Separate plan, implementation, critique and verification prompts; inspect the diff after each.
Verification evidence: The response names unverified assumptions and the commands actually executed.

Supervised vibe-coding loop

Human acceptance sits between every phase; generated code is a proposal until its behaviour and evidence pass.

01	02	03	04
Plan	Patch	Test	Review and retain

```
npm audit --audit-level=moderate && npm test && npm run build
```

Failure mode: A green build hides a wrong spatial contract, inaccessible control or unsupported-device dead end.
Control and recovery: Combine automated tests with runtime checks, console/network evidence and device capability truth.
Verification evidence: Plan matches diff; tests match acceptance criteria; runtime state matches the contract.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Storyboard as a state machine

Each frame names system state, user action and observable evidence—not merely a picture.

01	02	03	04
Entry	Orient	Act	Confirm

```
[{state:'Entry'}, {state:'Orient'}, {state:'Act'}, {state:'Confirm'}]
```

Failure mode: Four attractive screens omit plane detection, selection feedback or completion state.
Control and recovery: Make transitions conditional: unsupported → fallback; no plane → scan; hit → select; complete → confirm.
Verification evidence: Every transition has a trigger and visible result that can be tested.

World-space versus screen-space UI

World-space labels move with an object; screen-space controls remain stable relative to the viewport.

01	02	03	04
Camera view	World anchor	Screen overlay	Feedback channel

```
label.lookAt(camera.position);
```

Failure mode: A critical control drifts out of reach or a label occludes the real hazard it is meant to explain.
Control and recovery: Choose the coordinate space from the information lifecycle; keep safety actions screen-stable.
Verification evidence: Rotate the camera and verify labels retain context while controls remain reachable.

Angular size and viewing distance

Approximate angular size in radians as height ÷ distance for small angles; increasing distance halves apparent size.

01	02	03	04
Physical height	Viewing distance	Visual angle	Readable decision

```
angleRad = labelHeightMetres / distanceMetres
```

Failure mode: A label that looks readable on a laptop becomes illegible at arm's length in AR.
Control and recovery: Test at minimum/typical/maximum distance and set scale or distance constraints in the contract.
Verification evidence: A 4 cm label at 1 m produces about 0.04 rad (2.3°) and remains within the tested range.

Contrast over a live camera feed

Camera luminance changes continuously; a label needs an opaque/semi-opaque plate or outline, not a fixed text colour alone.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

01	02	03	04
Sample background	Measure luminance	Select surface	Retest in motion

```
material.depthTest = false; labelBackdrop.opacity = 0.86;
```

Failure mode: White text disappears over a bright wall; dark text disappears over machinery.
Control and recovery: Use a high-contrast plate, redundant icon/shape cues and real-environment testing.
Verification evidence: Labels remain readable over light and dark backgrounds without colour alone.

Depth ordering and occlusion

The depth buffer orders virtual fragments; real-world occlusion additionally requires depth/mesh data when supported.

01	02	03	04
Real geometry	Virtual depth	Depth test	Visibility decision

```
material.depthTest = true; material.depthWrite = true;
```

Failure mode: A virtual part draws through another object or hides the physical control surface.
Control and recovery: Declare the occlusion assumption and provide a non-occluding fallback when environment depth is unavailable.
Verification evidence: Near virtual surfaces occlude far ones; unsupported real-depth does not produce false claims.

Reachability and target bounds

Interaction uses a ray origin/direction and ordered intersections; visible size and collider size may differ.

01	02	03	04
Input ray	Target collider	Nearest hit	Feedback

```
const hit = raycaster.intersectObjects(selectables, true)[0];
```

Failure mode: The visible object is selectable only in a tiny area or the far object wins through an overlapping target.
Control and recovery: Use forgiving invisible hit geometry, nearest-valid-hit rules and immediate feedback.
Verification evidence: Three target distances return one nearest valid hit; empty space returns a visible no-hit state.

Equipment specifications into AR content

Extract physical dimensions, safe operating state, controls that must remain visible and the exact instruction content.

01	02	03	04
Envelope	Operating state	Hazard boundary	Information layer

```
scaleMetres: 0.65, callouts: 3, anchorPersistence: 'session'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Failure mode: A correctly modelled turbine is scaled wrongly or covers the emergency-stop control.
Control and recovery: Treat physical envelope and no-occlusion zones as acceptance constraints.
Verification evidence: Measured scale and unobstructed safety controls are captured in the review evidence.

Coordinate systems and metric units

Transforms compose from child to parent; WebXR reference spaces relate tracked poses to application world coordinates.

01	02	03	04
Model space	World space	Reference space	View space

```
worldMatrix = parent.matrixWorld * child.matrix
```

Failure mode: Centimetre-authored content interpreted as metres appears 100× too large.
Control and recovery: Normalise assets to metres at import and document the root scale.
Verification evidence: A 0.65 m source object measures 0.65 world units after import.

Device capability decision matrix

Capability is measured, not inferred from device brand; immersive modes may be independently supported.

01	02	03	04
Secure context	navigator.xr	Session support	Fallback route

```
await navigator.xr?.isSessionSupported('immersive-ar')
```

Failure mode: The AR button is shown on an unsupported desktop and ends in a dead click.
Control and recovery: Gate each mode separately and keep desktop preview available.
Verification evidence: Unsupported AR/VR presents the exact limitation and a working preview route.

Secure context and permission intent

Immersive sessions require secure-context APIs and a user gesture; tracking should start only after explicit intent.

01	02	03	04
Localhost or HTTPS	User activation	Session request	Consent duration

```
button.addEventListener('click', () => navigator.xr.requestSession('immersive-ar'))
```

Failure mode: The app requests spatial tracking on page load or suggests bypassing a browser security warning.
Control and recovery: Use localhost/HTTPS and a clear Enter AR/VR action; never

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

bypass safety interstitials.
Verification evidence: No tracking/session request occurs until the learner activates the immersive button.

XR session lifecycle

The session owns frame timing, input sources and reference spaces; session-end cleanup returns the app to desktop state.

01	02	03	04
Probe support	Request session	Render XR frames	End and clean

```
session.addEventListener('end', () => { hitTestSource = null; });
```

Failure mode: A stale hit-test source or controller state leaks into the next session.
Control and recovery: Register one lifecycle owner and reset session-scoped resources on `end`.
Verification evidence: Start–end–restart produces one fresh source and no duplicated event handlers.

Versioned scene schema

`ar-vr-vibe-scene/v1` is a contract; import rejects unknown schemas and invalid performance/accessibility fields.

01	02	03	04
Schema ID	Required fields	Validation	Migration/rejection

```
if (scene.schema !== 'ar-vr-vibe-scene/v1') throw new Error('Unsupported scene schema');
```

Failure mode: Silent import of an old or malformed scene fails later inside rendering code.
Control and recovery: Validate at the boundary and give a precise repair path.
Verification evidence: Valid v1 imports round-trip; unknown schema fails before scene mutation.

Topic 2: AR Application Development with Localhost WebXR

Three.js scene graph · WebXR sessions · hit test · anchors · raycast interaction

Alignment: K4 · K5 · K6 · A3

Three.js renderer ownership

One renderer owns the drawing buffer, XR session binding, pixel ratio and render loop.

01	02	03	04
Canvas	WebGLRenderer	XR manager	Animation loop

```
renderer.xr.enabled = true; renderer.setAnimationLoop(render);
```

Failure mode: Two animation loops compete or window.requestAnimationFrame stops during immersive presentation.
Control and recovery: Use `setAnimationLoop` as the single frame scheduler for

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

desktop and XR.
Verification evidence: Desktop and VR/AR modes render through one loop with no duplicate callback.

Scene graph transform chain

A child transform is evaluated relative to its parent; root scale and placement move every generated object coherently.

01	02	03	04
Scene	generatedRoot	Object group	Mesh

```
generatedRoot.position.setFromMatrixPosition(reticle.matrix);
```

Failure mode: Rotating the root also rotates callout anchors around the wrong pivot.
Control and recovery: Assign placement to the root and local animation to the intended child mesh.
Verification evidence: The root follows placement while child motion preserves anchor locations.

Perspective camera projection

The camera frustum maps view-space depth to clip space; tight near/far ranges improve depth precision.

01	02	03	04
Field of view	Aspect ratio	Near/far planes	Projection matrix

```
camera.aspect = width / height; camera.updateProjectionMatrix();
```

Failure mode: A resize stretches the scene or far/near values cause z-fighting and clipping.
Control and recovery: Update aspect on resize and choose the smallest practical depth range.
Verification evidence: A 390 px viewport preserves object proportions and no target is clipped at expected distance.

Lighting model and evidence

Hemisphere light supplies broad fill; a directional key defines form; shadows require renderer, light, caster and receiver settings.

01	02	03	04
Environment fill	Key direction	Material response	Shadow receiver

```
renderer.shadowMap.enabled = true; key.castShadow = true;
```

Failure mode: Enabling only one part of the shadow chain produces no shadow while code appears plausible.
Control and recovery: Verify the complete chain and maintain readable contrast under AR camera content.
Verification evidence: The object casts a visible, correctly oriented shadow on the floor preview.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Material contract

Physically based material parameters control reflected light; depthTest/depthWrite control virtual ordering.

01	02	03	04
Base colour	Roughness	Metalness	Depth policy

```
new THREE.MeshStandardMaterial({color, metalness:0.28, roughness:0.34})
```

Failure mode: Highly metallic content without an environment looks black or misleadingly glossy.
Control and recovery: Use restrained parameters and test against the actual lighting/background.
Verification evidence: Colour identity remains visible and surfaces retain form at multiple camera angles.

Geometry and draw-call cost

Each material/geometry group can create a draw call; instancing can submit repeated geometry together.

01	02	03	04
Geometry	Material	Mesh	Renderer submission

```
new THREE.InstancedMesh(geometry, material, count)
```

Failure mode: Dozens of identical callout meshes inflate draw calls and reduce mobile frame rate.
Control and recovery: Share geometry/materials or use instancing after measuring, not by assumption.
Verification evidence: Renderer stats remain within the 40-draw-call contract at the lab scene size.

Metric scale and asset normalisation

One world unit is treated as one metre; imported assets are normalised once at their boundary.

01	02	03	04
Authoring units	Import scale	Root normalisation	Runtime measure

```
generatedRoot.scale.setScalar(contract.content.scaleMetres);
```

Failure mode: Repeated scale corrections at parent and child levels multiply unexpectedly.
Control and recovery: Store real size in the contract and apply exactly one normalisation factor.
Verification evidence: Bounding-box measurement matches the equipment specification within tolerance.

Delta-time animation

Motion should depend on elapsed time, not frames, so behaviour remains consistent across refresh rates.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

01	02	03	04
Frame timestamp	Delta	State update	Render

```
rotation.y += radiansPerSecond * deltaSeconds;
```

Failure mode: A fixed per-frame increment runs 2× faster at 120 Hz than at 60 Hz.
Control and recovery: Use delta time and clamp long gaps after a suspended tab/session.
Verification evidence: One second produces the same angular change at 60 and 120 simulated frames.

Desktop orbit fallback

Desktop orbit is a review mode, not simulated immersive tracking; it exposes content when WebXR is unavailable.

01	02	03	04
Pointer input	Orbit controller	Camera target	Preview render

```
controls.enabled = !renderer.xr.isPresenting;
```

Failure mode: OrbitControls competes with XR headset camera pose during an immersive session.
Control and recovery: Disable desktop camera controls while XR is presenting.
Verification evidence: Desktop orbit works before/after XR and never overrides XR camera state.

AR session feature request

Required features make the request fail when absent; optional features enhance the session without blocking it.

01	02	03	04
Session mode	Required features	Optional features	DOM overlay

```
ARButton.createButton(renderer,{requiredFeatures:['hit-test'],optionalFeatures:['dom-overlay']})
```

Failure mode: Marking convenience features as required unnecessarily excludes capable devices.
Control and recovery: Require only mechanisms essential to the lab contract and provide fallback messaging.
Verification evidence: The request declares hit-test as essential and desktop preview remains reachable on failure.

Viewer-space hit-test source

A transient hit ray originates from viewer space; results are evaluated per XRFrame against the app reference space.

01	02	03	04
Viewer reference	Request source	Frame results	Pose

```
frame.getHitTestResults(hitTestSource)[0]?.getPose(referenceSpace)
```

Failure mode: Requesting a new hit-test source every frame wastes resources and may fail.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Control and recovery: Request once per session and reuse until the session ends.
Verification evidence: Instrumentation shows one source request per start–end cycle.

Reticle as placement evidence

The reticle shows the current placement pose before the learner commits content.

01	02	03	04
Hit result	Pose matrix	Reticle transform	Visible state

```
reticle.matrix.fromArray(pose.transform.matrix);
```

Failure mode: Content appears before a valid surface or the reticle remains visible after results disappear.
Control and recovery: Tie reticle visibility directly to current-frame hit results.
Verification evidence: No result hides reticle; stable result shows it at the detected surface.

Anchor and persistence boundary

A session-scoped placement remains stable only for the active XR session unless a persistent-anchor facility is explicitly available.

01	02	03	04
Pose selected	Root placed	Session tracks	Session ends

```
anchorPersistence: 'session'
```

Failure mode: The app promises persistence across reloads without storing or resolving a durable anchor.
Control and recovery: Name the persistence boundary in the scene contract and handover.
Verification evidence: Reload clearly resets placement and never claims cross-session persistence.

Raycast selection pipeline

Raycasting maps an input ray to distance-sorted scene intersections.

01	02	03	04
Pointer/controller ray	Selectable set	Ordered intersections	State feedback

```
const hit = raycaster.intersectObjects(selectables,true)[0] ?? null;
```

Failure mode: The ray tests the whole scene, selecting floor/helpers or hidden children.
Control and recovery: Maintain a named selectable collection and filter by state/layer.
Verification evidence: Nearest valid selectable object is highlighted; empty ray returns no-hit feedback.

Pointer coordinate normalisation

Pointer x/y must be relative to the rendering canvas, then mapped to [-1,1] with inverted y.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

01	02	03	04
Canvas bounds	Client coordinates	Normalised device coordinates	Camera ray

```
x=((clientX-left)/width)*2-1; y=-((clientY-top)/height)*2+1;
```

Failure mode: Using window dimensions produces mobile misses when the canvas is inset.
Control and recovery: Compute from the current canvas bounding rectangle for every pointer event.
Verification evidence: Touches at all four canvas corners map close to (-1,1), (1,1), (-1,-1), (1,-1).

XR controller input sources

Tracked-pointer rays originate from the controller or hand; handedness and targetRayMode are runtime properties.

01	02	03	04
Input source	Target ray	Select event	Haptic/visual feedback

```
const controller = renderer.xr.getController(0);
controller.addEventListener('select', place);
```

Failure mode: The implementation assumes a right-hand controller and fails for gaze or left hand.
Control and recovery: Use capability-driven input mapping and redundant feedback.
Verification evidence: Controller 0/1 or gaze input can trigger the same semantic action without hard-coded handedness.

Scene regeneration and disposal

Removing a mesh does not free its GPU buffers; geometry, materials and textures need disposal when no longer used.

01	02	03	04
Detach old graph	Traverse resources	Dispose GPU objects	Build new graph

```
child.traverse(o=>{o.geometry?.dispose(); o.material?.dispose();});
```

Failure mode: Repeated prompt variations grow GPU memory and eventually reduce performance.
Control and recovery: Centralise disposal at the regeneration boundary and avoid disposing shared resources prematurely.
Verification evidence: Ten regenerations show stable renderer memory counters and no console errors.

Scene export and import boundary

Only validated declarative data crosses the import boundary; executable code is never accepted from scene JSON.

01	02	03	04
Serialize contract	Download JSON	Validate input	Mutate scene

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
const scene = safeImport(await file.text());
```

Failure mode: An imported object contains unexpected code-like values or missing fields and partially mutates state.
Control and recovery: Parse, validate schema/checks, then apply atomically.
Verification evidence: Valid export round-trips; invalid schema leaves the previous scene unchanged.

Topic 3: Testing, Reviewing and Optimising AR/VR Applications

AR/VR boundary · comfort · frame budgets · diagnostics · handover

Alignment: K7 · A3 · A4

AR and VR runtime boundary

AR composites virtual content with the real environment; VR replaces the view and introduces play-area/comfort constraints.

01	02	03	04
Camera source	Reference space	Input model	Safety model

```
targetMode: 'ar' | 'vr'
```

Failure mode: A use-case comparison hides the implementation differences and reuses incompatible assumptions.
Control and recovery: Make the decision from environment, tracking, input, safety and performance contracts.
Verification evidence: A reviewer can name five concrete fields that change between AR and VR.

Reference-space choice

Reference spaces express different origin and movement assumptions; local-floor supplies floor-relative height where supported.

01	02	03	04
Viewer	Local	Local-floor	Bounded-floor

```
renderer.xr.setReferenceSpaceType('local-floor');
```

Failure mode: A seated user or uncalibrated floor gets an incorrect content height.
Control and recovery: Provide seated fallback and calibrate/bound the expected play space.
Verification evidence: Content origin and eye height match the selected reference-space contract.

Stereo frame budget

Frame budget in milliseconds is 1000 ÷ refresh rate: 16.7 ms at 60 Hz, 13.9 ms at 72 Hz, 11.1 ms at 90 Hz.

01	02	03	04
Refresh rate	Frame period	CPU update	GPU render

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
budgetMs = 1000 / targetFps
```

Failure mode: A 16 ms frame passes desktop 60 Hz but misses a 90 Hz headset deadline.
Control and recovery: Set the target from the actual device/session and measure CPU/GPU time separately.
Verification evidence: Measured total stays below the selected refresh-period budget.

Controller rays and affordance

A visible ray communicates reach and direction; targets need sufficient angular size and depth separation.

01	02	03	04
Ray origin	Ray visual	Target bounds	Selection feedback

```
controller.add(new THREE.Line(rayGeometry, rayMaterial));
```

Failure mode: An invisible ray makes controller selection feel random even when code works.
Control and recovery: Show ray/reticle feedback and enlarge hit bounds without misrepresenting the visible object.
Verification evidence: Learner can predict the selected target before pressing the controller trigger.

Teleport locomotion

Teleport moves the player rig to a valid floor point while maintaining head-relative height.

01	02	03	04
Aim parabola/ray	Validate floor	Preview target	Commit pose

```
if (hit.surface === 'walkable') rig.position.copy(hit.point);
```

Failure mode: Teleport lands inside geometry or continuous motion causes discomfort.
Control and recovery: Whitelist walkable surfaces, show destination, then commit on release.
Verification evidence: Invalid surfaces never accept; valid target shows preview and deterministic landing.

Guardian and physical safety

VR content must respect the platform play-area boundary; applications should avoid encouraging rapid movement near edges.

01	02	03	04
Known boundary	Virtual layout	Approach threshold	Warning/stop

```
guardian: true, seatedFallback: true
```

Failure mode: A required reach exceeds the play area or training ignores real obstacles.
Control and recovery: Design seated/standing variants and keep interactions within conservative reach.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification evidence: All required targets remain reachable without crossing the physical boundary.

Motion comfort controls

Artificial camera motion creates optic flow without matching vestibular input; teleport, snap turns and reduced motion reduce conflict.

01	02	03	04
Motion source	Acceleration	Visual flow	User control

```
reducedMotionToggle: true
```

Failure mode: Auto-rotation or smooth locomotion cannot be disabled.
Control and recovery: Expose motion control and default to the least provocative movement compatible with the task.
Verification evidence: Reduced-motion setting stops non-essential animation immediately.

Spatial accessibility

Use redundant colour/shape/text cues, reachable controls, readable labels and a non-immersive route.

01	02	03	04
Perceivable	Operable	Understandable	Fallback

```
highContrastLabels: true, seatedFallback: true
```

Failure mode: Meaning is encoded only by colour or requires a controller gesture unavailable to some users.
Control and recovery: Provide multiple feedback channels and keyboard/pointer desktop access.
Verification evidence: Core task can be completed from desktop and status is not colour-only.

FPS measurement

FPS = 1000 ÷ frame delta ms; a rolling mean reduces jitter but can hide spikes, so retain percentile/spike evidence when tuning.

01	02	03	04
Timestamp delta	Instantaneous FPS	Rolling window	Threshold decision

```
fps = 1000 / (time - lastTime);
```

Failure mode: A single 120 FPS reading is reported as stable performance.
Control and recovery: Use a representative window and record device, scene and mode.
Verification evidence: Rolling FPS meets target and no sustained frame exceeds the relevant budget.

Draw-call and triangle budgets

Triangles affect vertex/fragment work; draw calls add submission/state overhead. Both must be measured.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

01	02	03	04
Scene inventory	Renderer stats	Budget compare	Optimise/retest

```
renderer.info.render.calls; renderer.info.render.triangles;
```

Failure mode: The team lowers polygon count while draw-call overhead remains the actual bottleneck.
Control and recovery: Change one budget driver at a time and retest the same scene.
Verification evidence: Calls ≤ 40 and triangles ≤ 80,000 for the lab contract; FPS returns above target.

Memory-leak diagnosis

Stable scene regeneration should converge rather than monotonically grow geometry/texture/program counters.

01	02	03	04
Reproduce cycles	Observe counters	Find retained resource	Dispose and retest

```
console.table(renderer.info.memory);
```

Failure mode: A one-time warm-up is mistaken for a leak or an actual retained texture is missed.
Control and recovery: Compare a repeated fixed cycle after warm-up and trace ownership before disposal.
Verification evidence: Counters stabilise across repeated generate/reset cycles.

Console and network evidence

The first causal error often explains later failures; failed module/asset requests are runtime evidence, not cosmetic warnings.

01	02	03	04
Reproduce	Capture first error	Map to source/request	Verify recovery

```
npm run dev; open DevTools Console and Network
```

Failure mode: The team fixes the last cascading error or ignores a 404 that removed an AR asset.
Control and recovery: Start from a clean reload, capture timestamps/URL/status and fix the earliest cause.
Verification evidence: Reload produces zero unexpected console errors and zero failed required requests.

Responsive control surface

Immersive launch and fallback controls must remain usable on the device that hosts the AR session.

01	02	03	04
Viewport	Layout reflow	Touch targets	Overflow test

```
document.documentElement.scrollWidth <= document.documentElement.clientWidth
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Failure mode: A desktop three-column inspector overflows a 375 px phone or uses sub-44 px controls.
Control and recovery: Stack panels, preserve 16 px form text and measure tap targets/overflow.
Verification evidence: 375×667, 390×844 and 768×1024 show no horizontal overflow; actionable targets meet the touch rule.

Runtime defect taxonomy

Classify by observed subsystem before proposing fixes; one symptom may cross boundaries but needs a first testable hypothesis.

01	02	03	04
Tracking	Rendering	Interaction	Content/usability

```
defect = {expected, observed, evidence, subsystem, impact}
```

Failure mode: Drift is blamed on the 3D model when the actual issue is anchor/session lifecycle.
Control and recovery: Use subsystem evidence and rank user impact before editing.
Verification evidence: Every defect record names expected/observed state and a reproducible observation.

Root-cause review loop

A controlled review preserves the failing observation and changes one plausible cause at a time.

01	02	03	04
Reproduce	Isolate variable	Smallest change	Retest same evidence

```
test('reticle hides when no hit results', ...)
```

Failure mode: A broad rewrite removes the symptom but also changes behaviour and erases causality.
Control and recovery: Add a regression test before or with the bounded fix; keep a rollback path.
Verification evidence: The original failing case passes and adjacent tests remain unchanged.

Regression-test pyramid

Fast deterministic tests cover parsing/validation; browser tests cover UI; physical XR evidence covers device-only behaviour.

01	02	03	04
Pure contract tests	DOM interaction	Browser smoke	Supported-device XR

```
vitest run && vite build
```

Failure mode: A unit test is presented as proof that hit testing worked on a real device.
Control and recovery: Name the evidence boundary and list device checks separately.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification evidence: Automated and device-dependent results are reported without conflation.

Developer handover contract

Handover records the exact build, scene schema, constraints, evidence and unverified device assumptions.

01	02	03	04
Changed files	Rationale and risks	Commands/results	Rollback and open checks

```
git diff --check; npm audit; npm test; npm run build
```

Failure mode: A handover says 'working' without version, tests, or supported-device scope.
Control and recovery: Use a reproducible checklist and link evidence artifacts by relative path.
Verification evidence: A new developer can clone, install, run, verify and revert without hidden context.

Detailed Lab Guides

Lab 01 — Scene Brief and Prompt Contract

Duration: 60 minutes · Alignment: LO1 · K3 · A1

Translate a workplace requirement and equipment specification into a constrained, testable WebXR scene contract.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

1. Read `assets/equipment-spec.md`; highlight the physical size, operating context, safety boundary, and information the technician must see.
2. Start the shared localhost studio with `./start.command`; open the printed URL in a Chromium-based browser.
3. Paste the baseline request from `vibe-script.md` into the studio and select **Infer from prompt**.
4. Generate the scene, then inspect the JSON contract before judging the 3D preview.
5. Compare `targetMode`, `tracking`, `scaleMetres`, callout count, and performance budgets with the equipment specification.
6. Run the refinement prompt so the coding agent changes one contract rule at a time and preserves the schema.
7. Export the scene as `evidence/lab-01-scene.json`; capture the preview and Checks tab as screenshots.
8. Run `node verify.mjs` and retain the PASS output as assessment evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ schema is ar-vr-vibe-scene/v1
- □ target mode is AR
- □ subject is turbine
- □ three callouts are declared
- □ scale is expressed in metres

```
node labs/lab-01-scene-brief-and-prompt-contract/verify.mjs
```

Expected: The final output is PASS Lab 01.

Troubleshooting

- If npm is missing, install the current Node.js LTS release and reopen Terminal.
- If the viewport is blank, reload once and check the browser console for WebGL errors.
- If the inferred mode is wrong, make `AR` explicit or choose the AR override.

Cleanup

Stop the Vite server with Control-C. Keep only the exported JSON and evidence screenshots; do not commit `node_modules/` or `.env` files.

Detailed lab file: labs/lab-01-scene-brief-and-prompt-contract/README.md

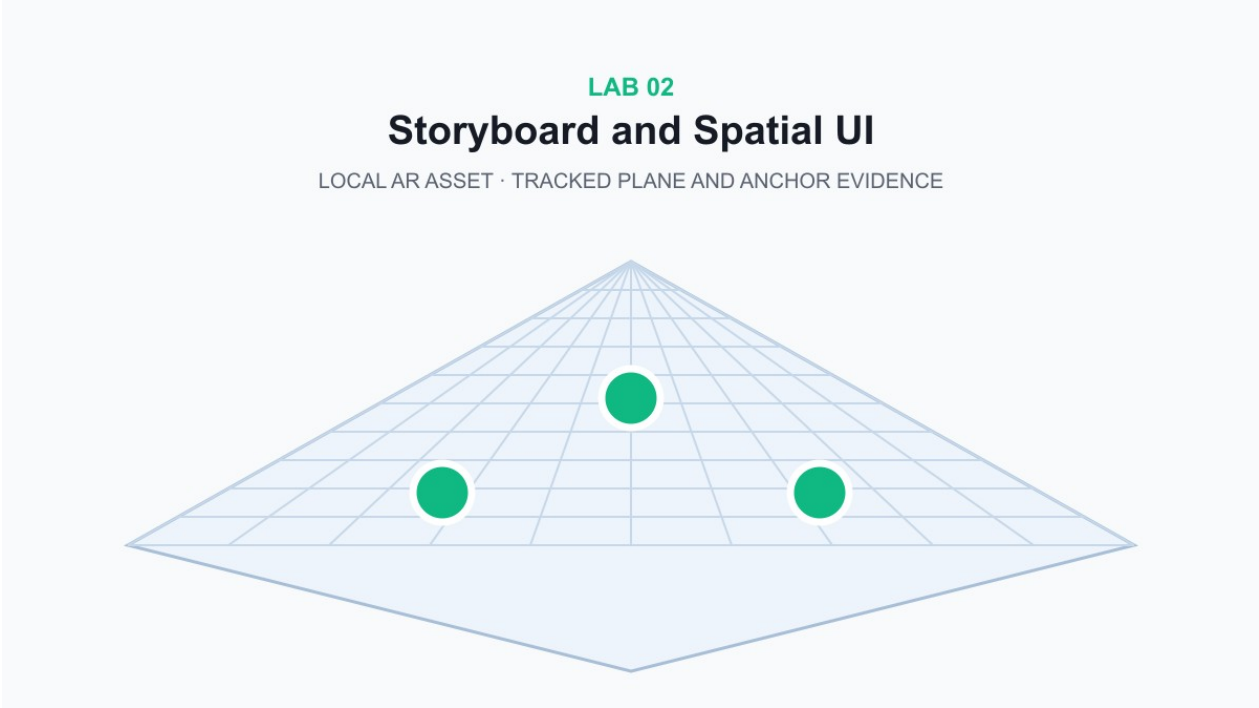
Lab 02 — Storyboard and Spatial UI

Duration: 75 minutes · Alignment: LO1 · K1 · K2 · A2

Convert an AR user journey into four observable states and apply readable, reachable spatial UI rules.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

9. Open `assets/storyboard-template.svg` and identify the Entry, Orient, Act, and Confirm evidence required in each frame.
10. Start the localhost studio and select the **Storyboard** inspector tab.
11. Run the baseline vibe prompt; confirm the generated sequence contains four distinct states rather than four decorative screens.
12. Check that Orient exposes plane detection/placement, Act names the interaction, and Confirm shows an observable completion signal.
13. Use the critique prompt to test label contrast, angular size, occlusion, reachability, and motion comfort.
14. Change one UI constraint, regenerate, and compare the previous and revised contract side by side.
15. Complete `assets/ui-review-checklist.md` with pass/fail evidence for each rule.
16. Export the scene and run `node verify.mjs`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ storyboard has four states
- □ high-contrast labels are enabled
- □ reduced-motion control is present
- □ tap interaction is declared

```
node labs/lab-02-storyboard-and-spatial-ui/verify.mjs
```

Expected: The final output is PASS Lab 02.

Troubleshooting

- If a storyboard frame is vague, rewrite it as a visible state plus user action plus evidence.
- If labels disappear, enable Spatial callouts in Runtime controls.
- If mobile text is cramped, use the 390 px responsive layout and verify there is no horizontal overflow.

Cleanup

Stop the server and keep the completed UI review plus exported scene in the evidence folder.

Detailed lab file: labs/lab-02-storyboard-and-spatial-ui/README.md

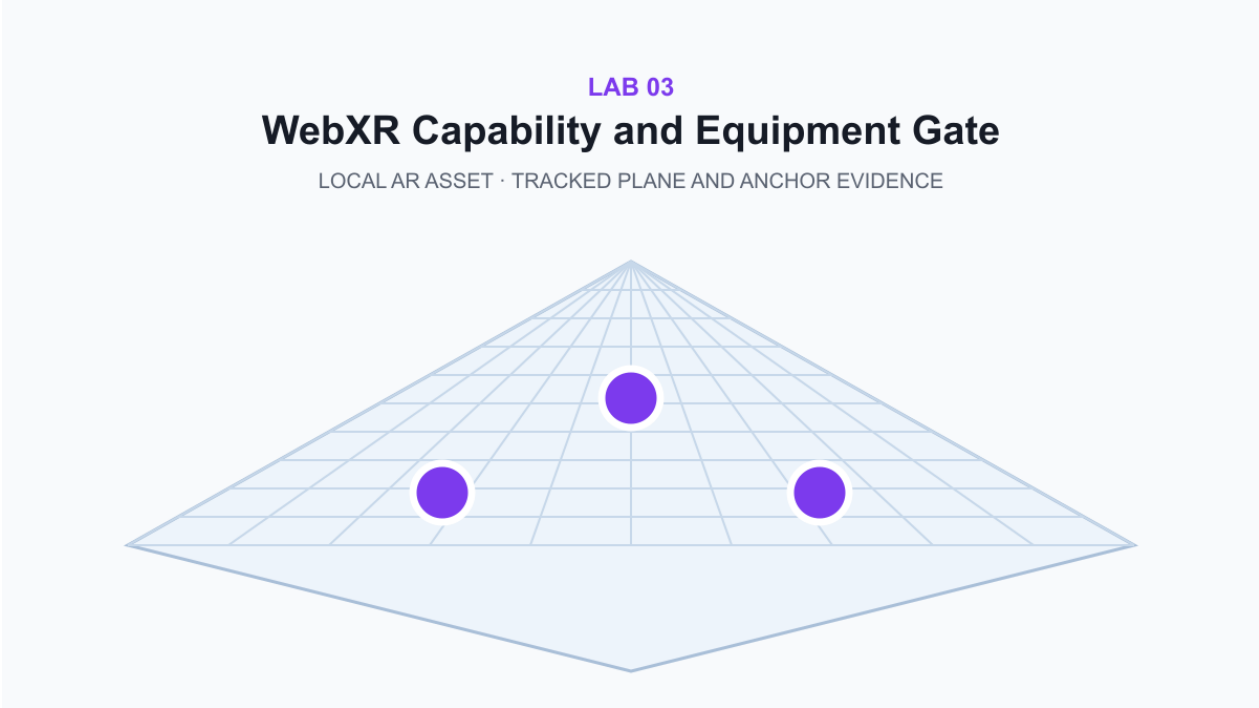
Lab 03 — WebXR Capability and Equipment Gate

Duration: 60 minutes · Alignment: LO1 · K3 · K5

Detect browser/device capability and route learners to AR, VR, or desktop fallback without a dead-end experience.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

17. Review `assets/capability-matrix.csv` and identify which combinations should enable AR, VR, or desktop preview.
18. Open the studio on desktop and record the actual AR/VR button states.
19. Use DevTools Console to inspect `window.isSecureContext`, `navigator.xr`, and WebGL renderer availability.
20. Run the vibe prompt and require a plan, minimal diff, and automated tests before accepting changes.
21. Simulate an unsupported browser condition and confirm the app keeps the desktop preview available.
22. Inspect the user message: it must name the missing capability and the safe next action without blaming cache.
23. Complete the capability decision table with observed device/browser evidence.
24. Run `npm test` in `webapp/` and `node verify.mjs` in this lab.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ desktop fallback remains available
- □ mode is explicit
- □ runtime budget is declared
- □ unsupported WebXR is communicated

```
node labs/lab-03-webxr-capability-and-equipment-gate/verify.mjs
```

Expected: The final output is PASS Lab 03.

Troubleshooting

- WebXR immersive sessions require HTTPS or localhost; use the exact local URL printed by Vite.
- Safari desktop may expose no immersive WebXR; use desktop preview for code review.
- If WebGL is disabled, confirm hardware acceleration before changing application code.

Cleanup

Close DevTools device simulation, stop the server, and retain the completed capability matrix.

Detailed lab file: labs/lab-03-webxr-capability-and-equipment-gate/README.md

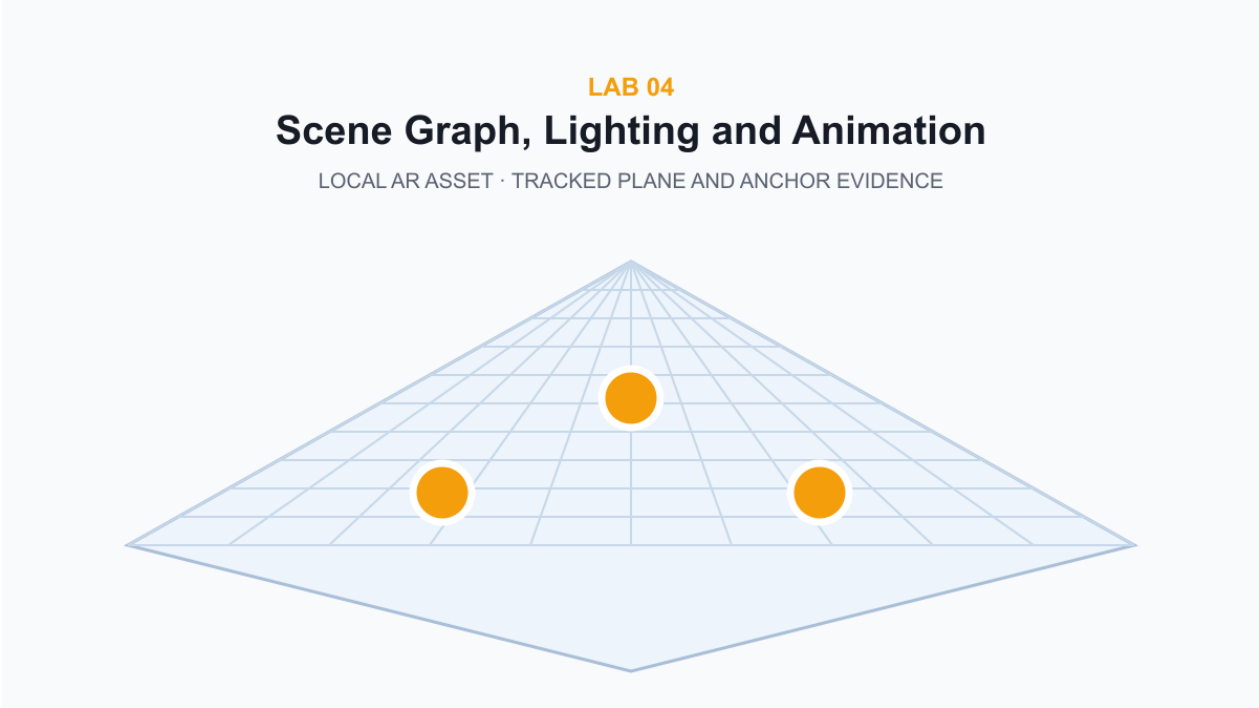
Lab 04 — Scene Graph, Lighting and Animation

Duration: 90 minutes · Alignment: LO2 · K4 · K5

Build a deterministic Three.js scene graph and explain how transforms, lighting, render loop, and cleanup interact.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

25. Trace `Scene → generatedRoot → Group → Mesh` in `assets/scene-graph.svg` and predict which transform affects every generated object.
26. Open `webapp/src/main.js`; locate scene creation, `generatedRoot`, `buildGeneratedScene()`, and `setAnimationLoop()`.
27. Run the vibe prompt and reject any response that replaces tested code without first explaining the object lifecycle.
28. Generate a three-object desktop scene and inspect the hierarchy through console logging or the provided contract.
29. Toggle motion and confirm the render loop changes behaviour without rebuilding the whole scene.
30. Generate ten variations and watch memory/console output for disposal errors or steadily increasing resources.
31. Record the before/after draw-call and FPS evidence in `evidence/runtime-review.md`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

32. Run the webapp tests, production build, and lab verifier.

Test it

- □ generated root uses metric scale
- □ animation can be disabled
- □ performance budget is explicit
- □ cleanup/disposal is documented

```
node labs/lab-04-scene-graph-lighting-and-animation/verify.mjs
```

Expected: The final output is PASS Lab 04.

Troubleshooting

- If every child rotates around the same origin, rotate the child mesh rather than the root group.
- If shadows vanish, verify the renderer shadow map, casting light, object castShadow, and floor receiveShadow as one chain.
- If repeated generation slows down, inspect disposal of geometry and materials.

Cleanup

Restore the baseline scene, stop the server, and keep the runtime review file.

Detailed lab file: labs/lab-04-scene-graph-lighting-and-animation/README.md

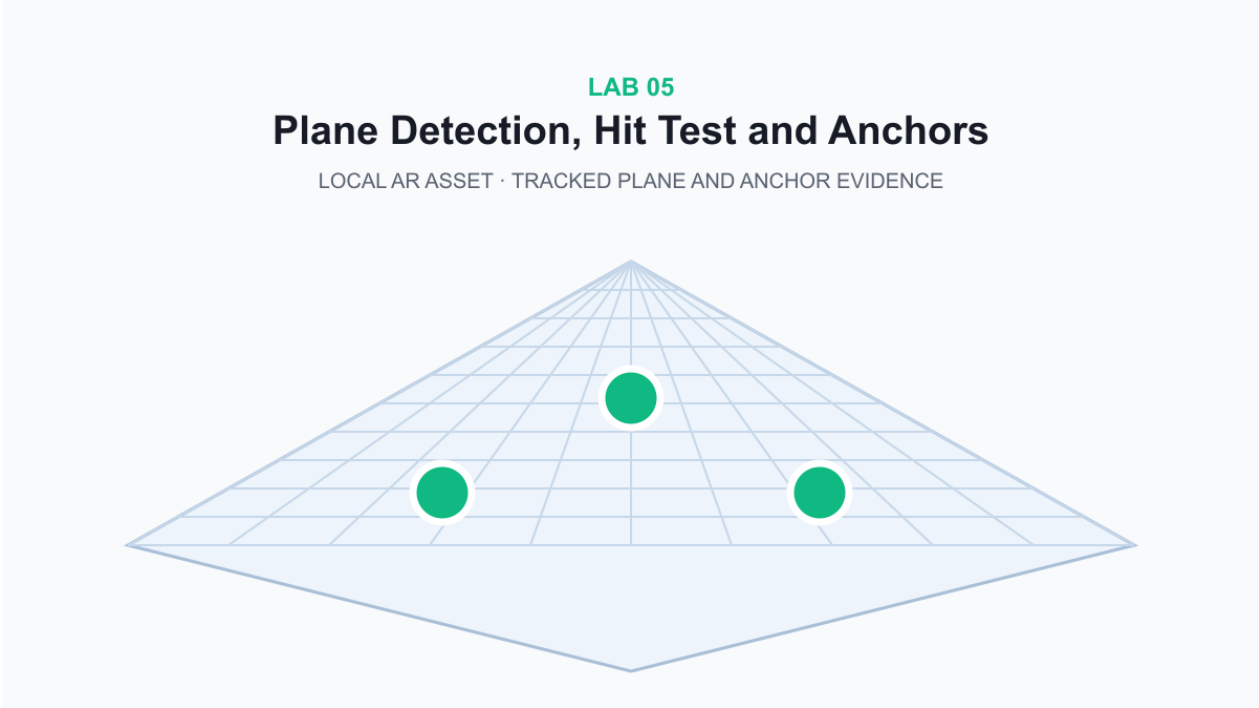
Lab 05 — Plane Detection, Hit Test and Anchors

Duration: 90 minutes · Alignment: LO2 · K5 · K6

Place virtual content on a detected real-world surface using viewer-space hit testing and a visible reticle.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

33. Study `assets/hit-test-lifecycle.svg`; identify session, viewer space, hit-test source, XRFrame, pose matrix, reticle, and placement states.
34. Locate `hitTestRequested`, `hitTestSource`, `reticle`, and the controller `select` handler in the app.
35. Run the vibe prompt and require the agent to preserve the one-request-per-session invariant.
36. On a supported device, enter AR, move the device until the reticle stabilises on a horizontal surface, then select to place.
37. On desktop, use the plane grid as evidence and review the same state machine without pretending hit-test support exists.
38. End and restart the AR session; verify the old hit-test source is cleared and a new one is requested.
39. Capture the reticle, placed object, and session-end evidence; annotate the pose/anchor relationship.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

40. Run `node verify.mjs`.

Test it

- □ AR uses horizontal-plane hit test
- □ reticle precedes placement
- □ session cleanup is explicit
- □ virtual object scale is metric

```
node labs/lab-05-plane-detection-hit-test-and-anchors/verify.mjs
```

Expected: The final output is PASS Lab 05.

Troubleshooting

- No reticle: improve lighting/texture on the physical surface and move the device slowly.
- Object drifts: avoid repeatedly overwriting the placed transform after selection.
- Session will not launch: confirm WebXR AR support and localhost/HTTPS secure context.

Cleanup

End the immersive session before stopping the server. Retain only non-sensitive room screenshots.

Detailed lab file: labs/lab-05-plane-detection-hit-test-and-anchors/README.md

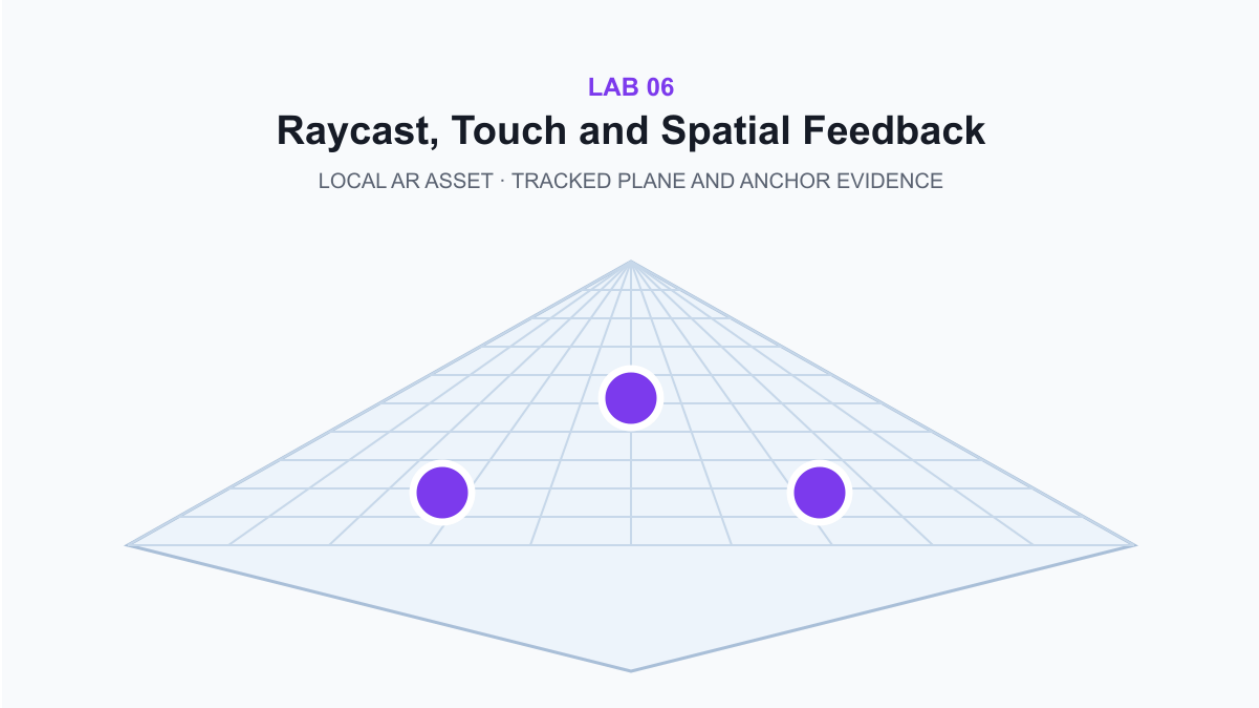
Lab 06 — Raycast, Touch and Spatial Feedback

Duration: 75 minutes · Alignment: LO2 · K2 · A3

Implement selection feedback and diagnose interaction misses using ray origin, direction, target bounds, and state evidence.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

41. Trace the interaction pipeline in `assets/raycast-pipeline.svg` from input coordinates to visible feedback.
42. Generate the four-object scene and predict which object should be selected for an overlapping ray.
43. Run the vibe prompt; check that the agent treats ray origin/direction, target collection, nearest hit, and feedback as separate states.
44. Test a successful hit, an empty hit, and two visually overlapping targets.
45. Disable spatial labels and verify selection still has a non-label feedback channel.
46. Use the Checks tab and console evidence to identify whether a miss came from coordinates, camera, layers, bounds, or target registration.
47. Document one defect and one adjustment in `evidence/interaction-review.md`.
48. Run automated and lab verification.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ tap interaction is declared
- □ nearest target is selected
- □ no-hit state is visible
- □ interaction review identifies a cause and adjustment

```
node labs/lab-06-raycast-touch-and-spatial-feedback/verify.mjs
```

Expected: The final output is PASS Lab 06.

Troubleshooting

- Wrong object selected: sort intersections by distance and use the first valid selectable target.
- Touch misses on mobile: calculate pointer coordinates from the canvas bounding rectangle.
- Invisible feedback: verify material state is restored when selection changes.

Cleanup

Restore default controls and keep the interaction review as assessment evidence.

Detailed lab file: labs/lab-06-raycast-touch-and-spatial-feedback/README.md

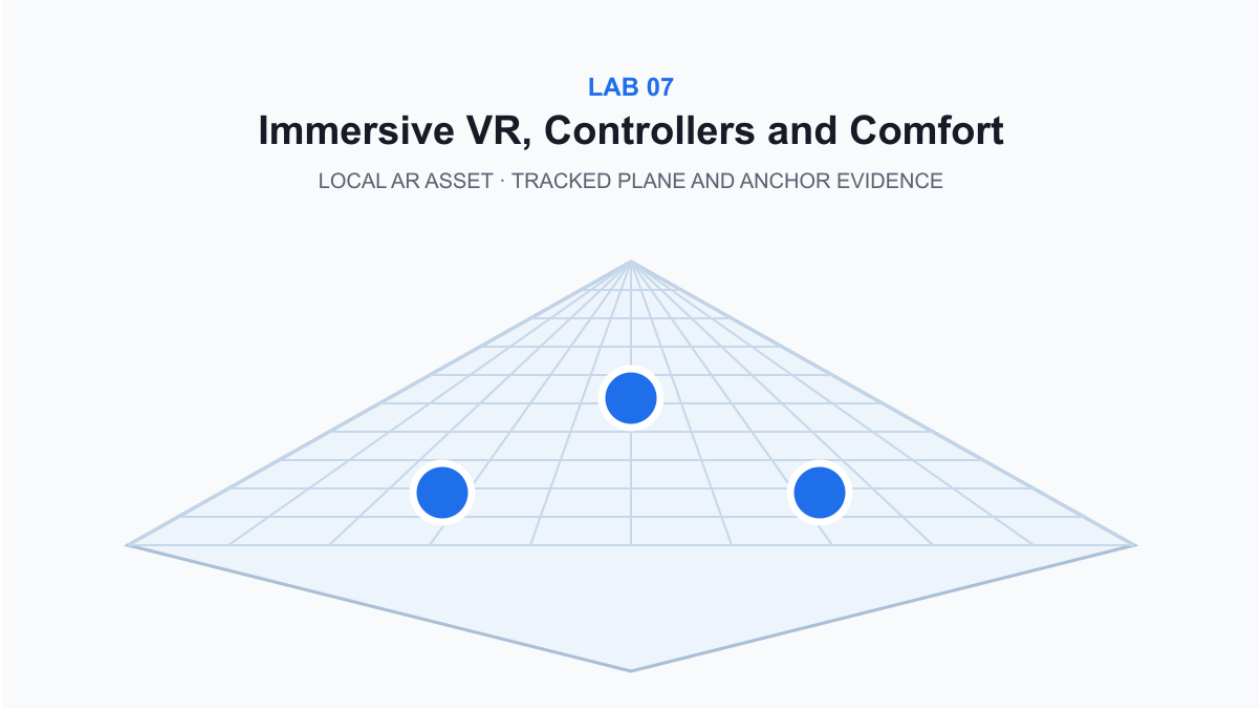
Lab 07 — Immersive VR, Controllers and Comfort

Duration: 75 minutes · Alignment: LO3 · K7

Convert the same content contract into a VR runtime and distinguish AR overlay constraints from VR room-scale constraints.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

49. Read `assets/ar-vr-decision-table.csv` and predict which requirements change when the real-world camera becomes a fully virtual environment.
50. Load the VR template and inspect `targetMode`, `environment`, reference-space origin, guardian, input, and frame-rate target.
51. Run the vibe prompt and require a plan that reuses content data while changing only runtime-specific contracts.
52. Enter VR on a supported headset or use desktop preview to review the controller/teleport contract honestly.
53. Verify seated fallback and reduced-motion controls are represented in the scene contract.
54. Complete the AR-versus-VR decision table with technical consequences rather than use-case slogans.
55. Export the VR scene and capture the contract plus runtime status.
56. Run `node verify.mjs`.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ target mode is VR
- □ local-floor origin is declared
- □ guardian and seated fallback are present
- □ target frame rate is at least 72 FPS

```
node labs/lab-07-immersive-vr-controllers-and-comfort/verify.mjs
```

Expected: The final output is PASS Lab 07.

Troubleshooting

- VR button disabled: verify `immersive-vr` support and the secure context.
- Floor height wrong: use local-floor reference space and calibrate the headset boundary.
- Discomfort: disable continuous motion and use teleport plus snap turns.

Cleanup

Exit VR, restore the physical play area, stop the server, and retain the completed decision table.

Detailed lab file: labs/lab-07-immersive-vr-controllers-and-comfort/README.md

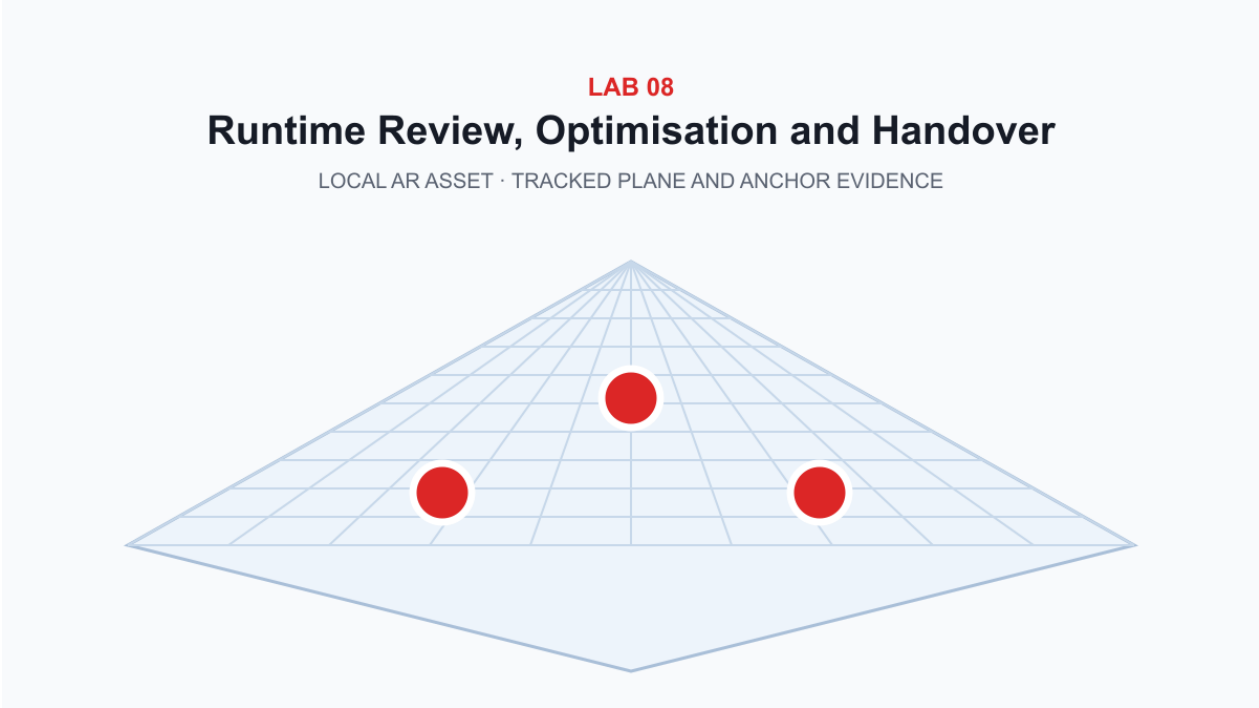
Lab 08 — Runtime Review, Optimisation and Handover

Duration: 90 minutes · Alignment: LO3 · A3 · A4

Review a generated AR/VR runtime, isolate defects with evidence, implement bounded adjustments, and prepare a reproducible handover.

Try this scene in the live Playground: https://alfredang.github.io/arvr-playground/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Workflow visual — tracked plane and anchor evidence. Detail is supplied in the lab asset folder.

Before you start

- Use a current Node.js LTS release and a Chromium-based browser.
- Keep credentials, learner data and private client material out of coding-agent prompts.
- Inspect every proposed diff and require automated plus runtime evidence.
- Clear a safe physical area before immersive testing.

Detailed procedure

57. Open `assets/runtime-review-template.md`; define the expected state, observed state, evidence, cause hypothesis, adjustment, and retest for each defect.
58. Start the app and create the five-object stress scene from the provided prompt.
59. Record FPS, console messages, failed network requests, interaction evidence, label readability, and responsive layout at 390 px.
60. Run the audit vibe prompt; reject broad rewrites or claims that lack observable evidence.
61. Select the highest-impact defect and implement the smallest bounded adjustment.
62. Add or update a regression test, then repeat the exact observation that originally failed.
63. Complete the handover with changed files, rationale, risks, test commands, results, and rollback path.
64. Run `npm audit`, `npm test`, `npm run build`, and `node verify.mjs`; capture all PASS outputs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

- □ all five contract checks pass
- □ one defect is evidenced
- □ one bounded adjustment is retested
- □ handover includes commands and rollback

```
node labs/lab-08-runtime-review-optimisation-and-handover/verify.mjs
```

Expected: The final output is PASS Lab 08.

Troubleshooting

- Low FPS: separate device limits from application load before lowering quality.
- Growing memory: verify disposal during scene regeneration.
- Mobile overflow: measure scrollWidth against clientWidth and fix the responsible element, not the whole layout.

Cleanup

Stop the server, remove generated `dist/` if rebuilding from source, and deliver the final JSON, screenshots, review, and test output without secrets.

Detailed lab file: labs/lab-08-runtime-review-optimisation-and-handover/README.md

Assessment Preparation

- Written Assessment (SAQ) — 7 open-ended questions, 60 minutes, open book.
- Practical Performance (PP) — 3 connected tasks, 90 minutes, open book.
- The current papers total 150 minutes; this conflicts with the generic 2-hour course-page label and is retained to mirror the approved paper-level evidence.

Source Register

Source	URL/path	Use	Checked/version
Official course page	https://www.tertiarycourses.com.sg/wsq-ai-vibe-coding-for-augmented-reality-ar-applications.html	Course identity, duration, outcomes, topics and current public description	11 September 2026
AR/VR Playground	https://alfredang.github.io/arvr-playground/	Learner-facing static HTML/CSS/JavaScript WebXR practice runtime	12 September 2026
W3C WebXR Device API	https://www.w3.org/TR/webxr/	Session modes, reference spaces, input sources,	9 June 2026 CRD

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		lifecycle, privacy and comfort	
W3C Immersive Web publications	https://www.w3.org/groups/wg/immersive-web/publications/	AR, hit-test, lighting and depth module status	11 September 2026
Three.js WebXRManager	https://threejs.org/docs/pages/WebXRManager.html	Renderer XR manager, sessions, controllers and reference space	11 September 2026
Three.js WebGLRenderer	https://threejs.org/docs/pages/WebGLRenderer.html	setAnimationLoop and renderer lifecycle	11 September 2026
Three.js ARButton	https://threejs.org/docs/pages/ARButton.html	Immersive AR button and session initiation	11 September 2026
Legacy v8 trainer deck	courseware/reference/WSQ - Master Trainer Slides - Develop Augmented Reality (AR) Applications - v8.pptx	Coverage floor: XR hardware, storyboarding, AR workflow, runtime review and enterprise work instructions	local source

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg